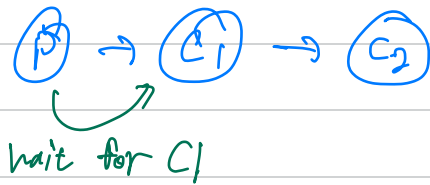


Q1

① Begin
Execution 1
End
Execution 2
End

(this is just one of the situation
where child runs first then
parent, by professor in lecture,
any valid order is OK!)

Q2: (C1 initial and 2 children)

① it has 3 process 

terminal output:

20
a: 10 ; c: 20

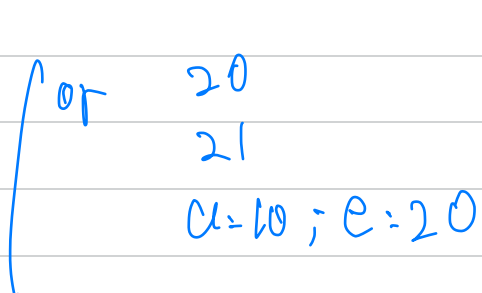
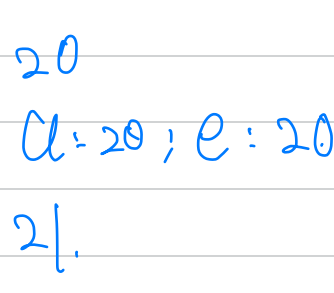
(C2 runs first, C1 then, P wait for C1)
C2 outputs nothing, C1 print 20,
P print a=10, and c received by C1=20

② still 3 process C1 init, 2 children

terminal output

21
20
a: 10 ; c: 20

(C2 runs first, C1 then, P wait for C1)
C2 outputs nothing, C1 print 20,
P print a=10, and c received by C1=20

or   (if other order)

Q?

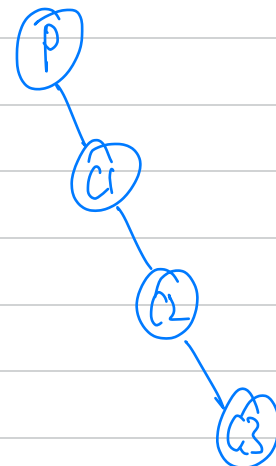
```
write_up > C Q2_3.c > main(void)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <unistd.h>
6
7  int main(void)
8  {
9      int a = 10;
10     int e;
11
12     if (fork() == 0) {
13         a *= 2;
14         if (fork() == 0) {
15             a += 1;
16             exit(2);
17         }
18         printf("%d\n", a);
19         exit(a);
20     }
21
22     sleep(30);
23     wait(&e);
24     printf("a:%d; e:%d\n", a, WEXITSTATUS(e));
25     return 0;
26 }
```

→ we added !

Q?

Q?

```
write_up > C Q3_1.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <unistd.h>
5
6  int main() {
7      int N = 3;
8
9      for (int i = 0; i < N; i++) {
10         pid_t pid = fork();
11
12         if (pid < 0) {
13             perror("fork");
14             exit(EXIT_FAILURE);
15         }
16
17         if (pid > 0) {
18             // parent stops creating more children
19             break;
20         }
21
22         // only child reaches here and continues the loop
23     }
24
25     return 0;
26 }
```



2

```

ite_up > C Q3_2.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <unistd.h>
6
7  // init process wait until the end of his child
8
9  int main() {
10     int N = 3;
11
12     for (int i = 0; i < N; i++) {
13         pid_t pid = fork();
14
15         if (pid < 0) {
16             perror("fork");
17             exit(EXIT_FAILURE);
18         }
19
20         if (pid > 0) {
21             // we are in the parent process, wait for the child process to finish
22             // check if the first child created or in other words, the init processm , if yes, then wait, else, go on
23             if (i == 0){
24                 waitpid(pid, NULL, 0); // wait for the first child process to finish
25             }
26
27             exit(EXIT_SUCCESS); // exit the parent process after waiting for the first child
28         }
29
30         // only child reaches here and continues the loop
31     }
32
33     return 0;
34 }
35
36

```

3

```

ep > C Q3_2.c > ...
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

// init process wait until the end of all process in the chain

int main() {
    int N = 3;

    for (int i = 0; i < N; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (pid > 0) {
            waitpid(pid, NULL, 0); // wait for the child process to finish
            exit(EXIT_SUCCESS);
        }

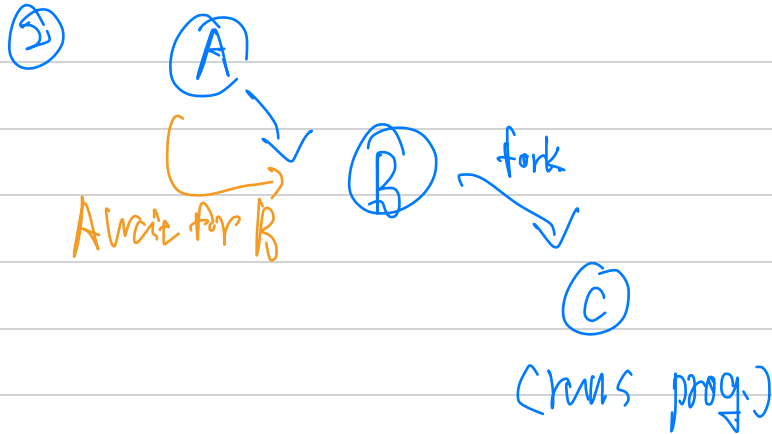
        // only child reaches here and continues the loop
    }

    return 0;
}

```

Q4

① In the child
In the parent.



when this code launch
it creates a process defined by A
then B process is forked
and A waits for B
B fork C process, and
exits

③

$C \rightarrow B \rightarrow A$

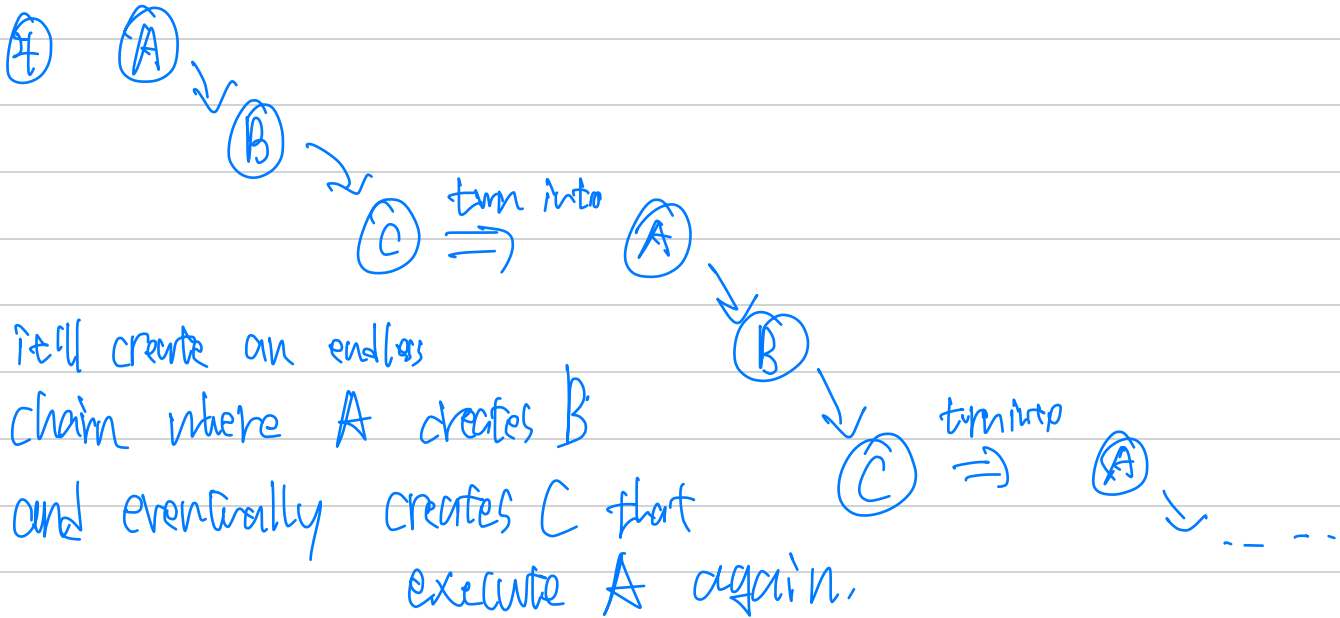
or

$B \rightarrow C \rightarrow A$

or

$B \rightarrow A \rightarrow C$

C then exec run another
program name "prog"



rite_up > C Q4_5.c > main(int, char * [])

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6
7  // file name : prog
8  // however, this prog will take one more argument, which is the count of previous launch.
9  // we will do so by a char of "0", "1", and "2"
10 // because the limit is at most 2 B, it is still acceptable
11 // the initial B_count will be "0"
12
13 int main(int argc, char *argv[]) {
14     int rcode;
15     char B_count = '0';
16     if (argc > 1) {
17         B_count = argv[1][0];
18     }
19
20     printf("%s\n", argv[0]); // A
21
22     if (B_count != '2') {
23         // can do the creation of B, because the limit is not reached yet
24         switch (fork()) {
25
26             case -1:
27                 perror("fork1()");
28                 exit(EXIT_FAILURE);
29
30             case 0: // B
31                 switch (B_count){
32                     case '0':
33                         B_count = '1';
34                         break;
35                     case '1':
36                         B_count = '2';
37                         break;
38                 }
39
40                 switch (fork()) {
41
42                     case -1:
43                         perror("fork2()");
44                         exit(EXIT_FAILURE);
45
46                     case 0: // C
47                         char B_count_str[2];
48                         B_count_str[0] = B_count;
49                         B_count_str[1] = '\0';
50
51                         if (execl("./prog", "prog", B_count_str, (char *)NULL) == -1) {
52                             perror("execl");
53                             exit(EXIT_FAILURE);
54                         }
55
56                     default:
57                         exit(EXIT_SUCCESS);
58                 }
59                 default:
60                     wait(&rcode);
61
62             }
63
64         }
65     }
66     else{
67         // B_count == '2', we have reached the limit, so we will not create any more child processes
68         exit(EXIT_SUCCESS); // no B branch created.
69     }
70
71
72     return EXIT_SUCCESS;
73 }
```

Since A eventually turns
into C, but exec 'l'
never put its value
into the launched program
so we let "prog" take
a B_count arg, then
it'll stop making the
third B and subsequently
C and a new A